

Captura guiada desde el celular: Plano 2D y Recorrido 3D

Documento de producto y técnico: especificación de punta a punta

Versión	v1.0 (borrador para implementación)
Fecha	29 de septiembre de 2026
Responsable de implementación y pruebas	Nico
Sponsor	Rony Bercovich
Sistemas involucrados	bercobackoffice (Django + React/Vite), re.bercovich.com (host público), EC2 + nginx, S3, nuevo worker GPU
Estado	Aprobado el enfoque. Falta validar la precisión en el piloto (sección 15)

Índice

- Resumen ejecutivo
- Qué existe hoy y qué hay que corregir primero
- Objetivos, no-objetivos y métricas de éxito
- Principios de diseño
- Experiencia de usuario de punta a punta
- Protocolo de captura (la especificación de "qué fotos")
- Arquitectura general
- Captura en el celular: especificación técnica
- Backend: modelos, API y carga
- Pipeline Fase 1: del set de fotos al plano 2D
- Pipeline Fase 2: recorrido virtual 3D
- Infraestructura y puesta en producción
- Observabilidad, logs y analítica
- Seguridad, privacidad y licencias
- Plan de pruebas y criterios de aceptación
- Plan de implementación por fases y ciclos de corrección
- Riesgos y mitigaciones
- Decisiones abiertas
- Anexos: referencias, textos de pantalla, checklists

1. Resumen ejecutivo

Queremos que un **dueño, solo, con su celular y sin instalar nada**, recorra su propiedad empezando en la puerta de entrada y saque las fotos que el sistema le va pidiendo. Con esa **única captura** generamos dos productos, en dos etapas:

- Fase 1: Plano 2D con medidas.** Plano por ambiente con dimensiones y m², superficie útil total y cubierta estimada. Se entrega en PNG/PDF y el dueño lo valida desde el mismo link.
- Fase 2: Recorrido virtual 3D.** Tour 360° navegable entre ambientes (estilo Matterport/Zillow), con vista "dollhouse". Más adelante, si la calidad lo permite, una reconstrucción fotorrealista (Gaussian splatting).

Condición de diseño: la captura de la Fase 1 ya incluye todo lo que la Fase 2 necesita. Guardamos originales y metadatos de sensores de cada foto. Cuando la Fase 2 esté lista, toda propiedad capturada se reprocesa **sin volver a la casa**.

El producto actual ("Owner Upload": link por WhatsApp para subir fotos y croquis por IA) queda como base de infraestructura: tokens, host público, notificaciones, revisión del asesor. Pero cambia la experiencia de captura y el motor de medición. Hoy el dueño recibe tres consejos y un selector de archivos, y el croquis sale de rectángulos estimados por un LLM ("NO A ESCALA"). Eso explica en buena parte por qué el link no se usa.

Stack propuesto (open source, uso comercial permitido): - captura web propia con cámara + giroscopio; - stitching 360 con OpenCV y poses del giroscopio; - layout de ambientes con **LGT-Net / HorizonNet** (MIT); - escala métrica con **MapAnything** (pesos Apache) o **Depth Anything 3 Metric** (Apache) más una referencia física (hoja A4 o medida de puerta); - tour con **Photo Sphere Viewer** (MIT); - dollhouse y splat con **gsplat** (Apache) y visor **Spark** (MIT).

2. Qué existe hoy y qué hay que corregir primero

2.1 El producto actual (Owner Upload, julio 2026)

Pieza	Dónde	Qué hace
Link opaco firmado	backend/properties/services/upload_links.py	Token django.core.signing (salt owner-upload-link), modos upload / review, UTM captacion_plano, texto WhatsApp "Te toma 90 segundos"
Modelos	backend/properties/models.py:956-1060	PropertyUploadLink (contadores uploads_count, bytes_total), PropertyAIAnalysis (status queued→running→draft→approved→returned→validated/failed, rooms JSON, floorplan_svg)
Página pública	backend/config/urls.py:42 (/p/<token>/) + backend/properties/templates/owner_upload/page.html	HTML + JS vanilla, <input type=file multiple>, 3 consejos, sube 1 archivo por POST (3 en paralelo, reintentos 1/3/9 s)
Guardas	backend/properties/services/owner_upload_guards.py	15 MB por imagen, 200 MB por video, 60 archivos / 300 MB por link, rate limit 30 req/min, detección de tipo por bytes mágicos
Pipeline IA	backend/properties/services/owner_ai_pipeline.py	Gate de calidad → clasificación de ambiente → agrupado → m² por "escalas de referencia" con LLM → cajas en grilla 0-100 → SVG. Motor: OpenAI gpt-4.1-mini (Gemini de backup)
Render	backend/properties/services/floorplan_render.py	Rectángulos con etiqueta "Living · 12.5 m² aprox." y leyenda "CROQUIS APROXIMADO — NO A ESCALA"
Job	backend/properties/management/commands/process_owner_analyses.py	Toma análisis queued y corre el pipeline
UI asesor	src/components/properties/OwnerUploadDialog.tsx, OwnerAIAnalysisSection.tsx	Crear/revocar link, revisar y aprobar el análisis, "enviar plano"
Tests	backend/properties/test_upload_links.py, test_owner_upload_public.py, test_owner_ai_pipeline.py, test_owner_expiry.py	

2.2 Problemas detectados en la revisión (se corrigen en la Fase 0)

1. **No hay cron para process_owner_analyses** en deploy/. Si en el EC2 tampoco existe, los análisis quedan en `queued` para siempre. **Verificar con crontab -l y ls /etc/cron.d.**
2. **nginx no tiene location para /p/** (deploy/nginx-bercobackoffice.conf): la ruta cae en el SPA. `public_host.py:28-33` lo marca como pendiente.
3. **client_max_body_size 50M** contra 200 MB de video permitidos: los videos grandes reciben un 413 de nginx.
4. **Permissions-Policy: camera=()** global. Bloquea `getUserMedia` en cualquier página que sirva ese nginx, y la nueva captura lo necesita.
5. **CSP: script-src 'self'**, sin `worker-src`. La captura va a usar Web Workers (control de calidad) y blobs. Hay que ajustarla para el host de captura.
6. **No hay telemetría.** Los `gtag` de `page.html` no hacen nada (no hay ID de GA4, TODO en `page.html:32`) y la apertura del link no se registra en el servidor. **Hoy no se puede saber cuántos dueños abren el link.**
7. **Contrato roto front↔back:** el backend devuelve `rooms` como dict `{rooms:[...snake_case], adjacencies,...}` y el front (`src/types/owner-upload.ts:52-72`, `OwnerAIAnalysisSection.tsx:86`) espera un array `camelCase`. La tabla de ambientes no se ve, y guardar pisa el dict con un array.
8. **Regenerar incluye el plano anterior como foto del dueño:** `owner_ai_pipeline.py:677-681` excluye frames de video, pero no `alt='Plano generado'`.
9. `complete/` crea una versión nueva de análisis en cada toque de "enviar", sin `debounce`. `rejected` siempre devuelve 0 aunque la UI diga lo contrario.
10. `cairosvg` no está instalado: el PNG del plano usa el fallback de Pillow.

2.3 Qué se reutiliza y qué se reemplaza

Se reutiliza	Se reemplaza o se agrega
Token firmado, host público, revocación, expiración, mensaje WhatsApp, UTM	Página de captura (nueva app web de cámara guiada, en vez del <code><input type=file></code>)
PropertyAIAnalysis y su máquina de estados, notificaciones al asesor, flujo <code>send-plan → validate</code>	Modelo de datos de captura (sesión, ambientes, tomas con metadatos)
<code>image_pipeline.generate_variants</code> para miniaturas	Carga: de POST multipart a Django a subida directa a S3 por partes
Stage 0/1 del pipeline LLM (gate de calidad y tipo de ambiente) como señal auxiliar	Estimación de m² y layout: de LLM a geometría (layout 360 + escala métrica)

Se reutiliza	Se reemplaza o se agrega
OwnerAIAnalysisSection (arreglada) como pantalla de revisión	Render del plano: de cajas en grilla a polígonos con medidas y escala real

3. Objetivos, no-objetivos y métricas de éxito

3.1 Objetivos

- **O1.** Un dueño sin conocimientos técnicos completa la captura de un 3 ambientes en ≤ 15 minutos, sin instalar nada, desde iPhone (Safari) o Android (Chrome).
- **O2.** El plano 2D tiene error de superficie $\leq 5\%$ **por ambiente** (mediana) y $\leq 3\%$ **en el total útil** frente a una medición láser (objetivo del piloto; el umbral de salida está en la sección 15).
- **O3.** La captura de Fase 1 alcanza para producir el tour 3D de Fase 2 sin volver a capturar.
- **O4.** La carga funciona con señal mala, fotos pesadas o livianas y cortes: **cero fotos perdidas** una vez sacadas.
- **O5.** Hay métricas por paso de todo el embudo, del link enviado al plano validado.

3.2 No-objetivos (por ahora)

- Planos con validez legal o catastral: el plano es informativo y lo aclara.
- App nativa o LiDAR: queda como "modo preciso" opcional (sección 18).
- Exteriores, terrenos, fachadas, casas de varias plantas. Se admite un nivel; los dúplex van como dos capturas.
- Amoblamiento virtual o home staging.

3.3 Métricas

Métrica	Definición	Meta piloto	Meta producción
Tasa de apertura	links abiertos / links enviados	medir	$\geq 60\%$
Tasa de inicio	"Empezar" / links abiertos	medir	$\geq 50\%$
Tasa de finalización	captura completa / iniciadas	$\geq 60\%$	$\geq 70\%$
Tiempo de captura	inicio $\rightarrow$ fin, 3 ambientes	≤ 20 min	≤ 15 min
Fotos rechazadas en vivo	% de tomas que la app pide repetir	medir	$\leq 15\%$
Error de área por ambiente	mediana abs(%) contra láser	$\leq 6\%$	$\leq 5\%$
Error de área total útil	abs(%) contra láser	$\leq 4\%$	$\leq 3\%$
Tiempo a plano en borrador	fin de captura $\rightarrow$ draft	≤ 30 min	≤ 15 min
Validación del dueño	"Sí, está bien" / planos enviados	medir	$\geq 70\%$
Costo de cómputo	USD por propiedad (F1 / F2)	medir	$\leq 0,30 / \leq 2$

4. Principios de diseño

1. **Una captura, dos productos.** Cada toma se pide porque sirve al plano, al 3D o a ambos (sección 6). Nunca guardamos solo derivados: originales y metadatos siempre.
2. **El dueño no tiene que saber nada.** La app decide cuándo disparar (disparo automático al alinear y quedarse quieto), qué falta y qué repetir. Los textos hablan de "girar", "apuntar", "caminar", nunca de "panorama" o "paralaje".
3. **Nunca se pierde una foto.** Cada foto se guarda primero en el celular (IndexedDB) y después se sube. La sesión es reanudable desde el mismo link, en el mismo o en otro momento.
4. **Tolerancia a la calidad.** Aceptamos lo que el dispositivo pueda dar, avisamos si es malo y marcamos la confianza. No bloqueamos al dueño salvo que la foto sea inutilizable (negra, totalmente movida).
5. **Degradación elegante.** Si no hay cámara integrada o sensores (permiso denegado, navegador viejo), se ofrece un modo simple con la cámara nativa y guía por texto. Menos precisión, pero el dueño termina.
6. **El humano en el medio.** El asesor revisa y aprueba antes de que el dueño vea el plano, igual que hoy.
7. **Medible desde el día uno.** Cada pantalla emite un evento al servidor.

5. Experiencia de usuario de punta a punta

5.1 Actores

- **Asesor** (backoffice): genera el link, lo manda, recibe avisos, revisa y aprueba el plano y el tour.

- **Dueño** (celular): captura, valida el plano, pide tasación.
- **Revisor interno** (opcional, puede ser el asesor): corrige el plano en el editor antes de aprobarlo.
- **Sistema**: procesa, notifica y re-procesa.

5.2 Paso A: el asesor genera el link (backoffice)

1. En la ficha de la propiedad (`FichaPropiedad / PropertyDetail / AcervoSection`), botón "**Pedir plano y recorrido al dueño**". Reemplaza al actual "Link de carga".
2. Diálogo (evolución de `OwnerUploadDialog.tsx`): - Nombre del dueño (precargado del contacto). - Dirección (precargada de la propiedad; editable). - Tipo: departamento / PH / casa (precargado). - Cantidad de ambientes y m² de escritura (precargados si existen; opcionales). - Vencimiento: 7 / 14 / 30 días.
3. El sistema crea la sesión de captura, devuelve la URL pública `https://re.bercovich.com/c/<token>` y un mensaje de WhatsApp listo para copiar o abrir en `wa.me`:

"Hola {nombre}, te paso el link para armar el plano de tu propiedad con tu celular. Te guía paso a paso, lleva unos 15 minutos y lo podés pausar cuando quieras. Conviene hacerlo de día con las luces prendidas: {url}"

4. En la ficha aparece el estado de la sesión en vivo: *Enviado* → *Abierto* → *En curso (3/6 ambientes)* → *Completo* → *Procesando* → *Plano listo para revisar*.

5.3 Paso B: el dueño captura (celular). Pantalla por pantalla

La numeración P0-P12 es la que usa el diseño, los eventos de analítica (sección 13) y los tests E2E.

P0 • Bienvenida - Logo, "Hola {nombre}", dirección de la propiedad. - "Vamos a armar el plano de tu propiedad con tu celular. Te vamos a guiar paso a paso: **unos 15 minutos**." - Qué se necesita: luces prendidas, cortinas abiertas, puertas interiores abiertas, 15 minutos. Opcional: un metro o una hoja A4. - Botón "**Empezar**". Link secundario "¿Cómo funciona?" con un video de 30 s. - Si la sesión ya tiene progreso: "**Seguir donde quedaste (3 de 6 ambientes)**".

P1 • Datos mínimos (una sola pantalla, todo opcional salvo confirmar la dirección) - Confirmar dirección (precargada) y piso/depto. - ¿Cuántos ambientes tiene? (1 / 2 / 3 / 4 / 5+ / No sé) - ¿Sabés los m² totales? (campo numérico, "No sé") - ¿Tiene balcón, patio o terraza? (chips) - Con esto se genera la **lista de ambientes sugerida**: living-comedor, cocina, N dormitorios, baño(s), lavadero, balcón. El dueño la puede editar en P3.

P2 • Permisos y chequeo del dispositivo - Un botón "**Activar cámara**". En el mismo toque se piden: - `getUserMedia` (cámara trasera); - en iOS, `DeviceOrientationEvent.requestPermission()` y `DeviceMotionEvent.requestPermission()`, que requieren gesto del usuario. - Chequeos automáticos: - la cámara abre y se mide la resolución real; - llegan eventos del giroscopio; - hay espacio en el almacenamiento (`navigator.storage.estimate()`); - se pide almacenamiento persistente (`navigator.storage.persist()`). - **Si falla la cámara integrada** → modo simple (sección 8.7). - **Si fallan los sensores** → modo asistido sin disparo automático: el dueño toca para sacar y la guía es por texto y ángulos aproximados.

P3 • Lista de ambientes - Tarjetas con los ambientes sugeridos. Cada una tiene "+ agregar", renombrar y quitar. - Arriba: "Empezamos en la **puerta de entrada**. Andá hasta ahí." - Botón "Estoy en la puerta".

P4 • Toma de entrada (puerta de entrada) - Visor de cámara a pantalla completa con un marco guía: "Parate **adentro**, de espaldas a la puerta de entrada, y apuntá hacia el interior." - Disparo automático cuando el teléfono está vertical (pitch $\approx 0^\circ \pm 10^\circ$, roll $\pm 5^\circ$) y quieto 400 ms. - Después: "Ahora girate y sacale una foto a la puerta de entrada." Esta segunda toma registra dónde está la puerta en el plano.

P5 • Elegir ambiente actual - "¿En qué ambiente estás?" con chips de la lista (el primero ya sugerido). "Parate en el **centro** del ambiente."

P6 • Giro 360° guiado (la toma principal) - Instrucción animada: "Sostené el celular vertical a la altura del pecho y **girá sobre tu lugar, despacio**, como una calesita. Movés los pies, no el brazo." - Anillo en pantalla con **8 objetivos** (cada 45° de yaw, relativos al yaw inicial) y un punto que muestra la orientación actual. - **Disparo automático** cuando: 1. el punto entra en el objetivo ($\pm 6^\circ$); 2. pitch está en rango ($\pm 10^\circ$) y roll en $\pm 5^\circ$; 3. la velocidad angular es menor a $15^\circ/\text{s}$ durante 300 ms; 4. el control de calidad rápido pasa (sección 8.6). - Feedback: vibración corta (`navigator.vibrate`, solo Android) y "clic" visual, el objetivo se pone en verde y aparece el contador "5/8". - Avisos en vivo, uno a la vez y con prioridad: "Más despacio", "Enderezá el celular", "Está muy oscuro: prendé la luz", "Salió movida, quedate quieto un segundo". - **Ambientes grandes** (el dueño marca "es grande" o el living tiene > 5 m según la estimación en vivo, sección 18): se pide un **segundo giro** en otro punto del ambiente. - **Opcional recomendado (fila superior)**: 4 tomas extra apuntando 30° hacia arriba para ver la unión pared-techo. Mejora el layout y el 360. Se activa por configuración según el piloto.

P7 • Esquinas - "Ahora andá a un **rincón** del ambiente y apuntá al **rincón de enfrente**." Se repite para 4 rincones (2 en baños y pasillos). - El visor muestra una cruz. Disparo automático con el teléfono quieto y nivelado. - Si el ambiente es muy chico (baño), se permite "No puedo alejarme más" y se sigue.

P8 · Referencia de escala (una vez por propiedad, en el primer ambiente con piso visible) - Opción A (recomendada): "Poné una **hoja A4** en el piso, en el centro, y sacale una foto desde arriba y otra desde donde estás parado." La hoja queda también en el giro 360 si se hace antes: el orden en la UI es escala → giro. - Opción B: "Medí con un metro el **ancho de una puerta** (de marco a marco) y escribilo." Luego una foto de esa puerta. - Opción C: "No tengo ninguna": se usa solo la profundidad métrica (más error, se marca confianza baja).

P9 · Puerta de paso - "¿A qué ambiente vas ahora?" (chip). "Parate en el **marco de la puerta** y sacá una foto hacia el ambiente que dejás y otra hacia el que entrás." - Estas dos tomas conectan los ambientes (grafo de adyacencias) y en la Fase 2 son los "saltos" del tour. - Vuelve a P5 con el siguiente ambiente.

P10 · Revisión de la captura - Mini-mapa esquemático: cajas por ambiente conectadas por las puertas registradas, con ✓, △ ("faltan 2 esquinas") o X. - Barra de carga global: "Subidas 74 de 86 fotos". Si hay fotos pendientes: "Podés cerrar: seguimos subiendo cuando vuelvas a abrir el link." Ver la limitación de iOS en la sección 8.5. - Botones: "Rehacer ambiente", "Agregar ambiente", "**Terminé**".

P11 · Cierre - "¡Listo! Estamos armando tu plano. Te avisamos por WhatsApp cuando esté (normalmente en menos de 1 hora)."
- **Momento wow inmediato:** si el 360 del primer ambiente ya se procesó en vivo (sección 10.2, paso 3), se muestra ese ambiente navegable arrastrando el dedo: "Mirá cómo quedó tu living en 360°."

P12 · Revisión del plano (modo `review`, mismo link, cuando el asesor lo envía) - Plano con medidas por ambiente, m² por ambiente y totales (útil y cubierta estimada, con leyenda). - "Descargar plano (PDF)". - "¿Está bien?": - "Sí, está bien"; - "Hay algo distinto" (texto de 280 caracteres + tocar el ambiente que está mal). - "Solicitar tasación" (flujo actual `valuation/`). - **Fase 2:** botón "**Ver recorrido 3D**" cuando esté disponible.

5.4 Errores y recuperación (UX)

Situación	Comportamiento
Sin señal durante la captura	La captura sigue normal. Indicador "sin conexión, guardando en el celular". Sube al volver la señal
El dueño cierra el navegador	Al reabrir el link: "Seguir donde quedaste" + reanuda las subidas pendientes desde IndexedDB
Cambia de celular	La sesión sigue, pero las fotos no subidas del otro celular quedan allá. P10 muestra qué ambientes quedaron incompletos
Almacenamiento del celular lleno	Se baja la calidad local (se guarda solo la versión de trabajo, sección 8.4) y se avisa
Batería < 15%	Aviso: "Te queda poca batería, conectalo o seguí después"
Permiso de sensores denegado	Modo asistido (disparo manual)
Cámara integrada no disponible	Modo simple con <code><input capture></code> (sección 8.7)
Link vencido o revocado	Pantalla "Este link venció. Pedile uno nuevo a {asesor}" + botón WhatsApp al asesor
Foto rechazada por el control de calidad	Se pide repetir en el momento; nunca se descarta en silencio. Tras 3 intentos se acepta con marca de baja calidad

5.5 Paso C: el asesor revisa (backoffice)

- Notificación "Captura completa de {dirección}" (nuevo kind `CAPTURE_COMPLETE`), y después "Plano en borrador listo" (`OWNER_ANALYSIS_DRAFT` existente).
- Pantalla de revisión (evolución de `OwnerAIAnalysisSection.tsx`): - Plano SVG a la izquierda y tabla de ambientes a la derecha: tipo, ancho × largo, m², confianza, fotos. - Indicadores de confianza por ambiente (verde/amarillo/rojo) y por total. - **Editor mínimo:** renombrar ambiente, corregir tipo, **editar una medida**. Al cambiar una cota, el sistema re-escala ese polígono y **re-renderiza el SVG**; hoy el SVG no se regenera al editar. Mover o eliminar puertas. - Galería por ambiente con las tomas (giro, esquinas, puertas) para chequear. - Botones "Aprobar y enviar al dueño" (→ `send-plan`), "Reprocesar", "Pedir que rehaga un ambiente". Este último envía al dueño un link directo a P5 de ese ambiente.
- Cuando el dueño valida o pide cambios, llegan las notificaciones actuales (`OWNER_FEEDBACK`).

5.6 Paso D (Fase 2): entrega del recorrido

- El asesor ve "Recorrido 3D listo" y lo previsualiza; al aprobar se publica en:
- la **ficha pública** de la propiedad (embed);
- el link del dueño (P12 "Ver recorrido 3D");
- una URL compatible `re.bercovich.com/t/<slug>` para portales y redes.

6. Protocolo de captura: qué fotos y para qué sirve cada una

6.1 Tipos de toma

Código	Toma	Cantidad	Cómo	Usa Fase 1 (plano)	Usa Fase 2 (3D)
entry_in / entry_door	Entrada	2 por propiedad	Desde adentro hacia el interior + hacia la puerta	Ancla del plano, ubicación de la puerta	Nodo inicial del tour
pano_k (k=0..7)	Giro 360	8 por punto de giro	Desde el centro, cada 45° de yaw, vertical, a la altura del pecho	Panorama → layout (paredes, esquinas del ambiente)	Foto 360 del nodo del tour; textura del dollhouse
pano_up_k (opcional)	Giro superior	4	+30° de pitch	Mejora el borde pared-techo del layout	Completa el cenit del 360
corner_k	Esquinas	4 (2 en baño y pasillo)	Desde un rincón apuntando al opuesto	Triangulación multi-vista: corrige la escala y la forma del layout	Paralaje para la profundidad, el dollhouse y el splat
door_from / door_to	Puerta de paso	2 por puerta	Desde el marco, a cada lado	Adyacencias y alineación entre ambientes	Enlaces entre nodos del tour ("saltos")
scale_top / scale_ctx	Escala	2 por propiedad	A4 en el piso: cenital + contexto	Escala métrica absoluta	Escala del 3D
scale_door + medida	Escala alternativa	1 + número	Foto de la puerta medida	Escala métrica absoluta	Escala del 3D
extra	Libre	0-10	El dueño agrega (detalles, vistas)	Clasificación de estado (pipeline LLM existente)	Fotos de galería

Volumen típico: un 3 ambientes (living-comedor, cocina, 2 dormitorios, baño, lavadero, balcón = 7 espacios) lleva unas 7 × (8 + 4) + 6 × 2 (puertas) + 4 ≈ **100 tomas**. El piloto dirá si hace falta recortar, por ejemplo con 6 objetivos en baños o 2 esquinas en ambientes chicos.

6.2 Por qué este protocolo alcanza para las dos fases

- **Plano:** el giro 360 da la forma del ambiente con modelos de layout probados (HorizonNet/LGT-Net). Las esquinas dan una segunda vista para corregir la escala y la forma. Las puertas permiten unir ambientes. La A4 o la puerta medida fija los metros.
- **Tour 360:** cada giro es un nodo y cada par de puertas un enlace. No hace falta nada más.
- **Dollhouse:** el layout ya es geometría 3D (piso, paredes, altura); se texturiza con los panoramas.
- **Splat fotorrealista:** necesita fotos desde distintos lugares (paralaje). El giro solo no alcanza porque es rotación pura, pero **las esquinas y las puertas sí la aportan**. Si el piloto muestra que no alcanza para un splat de calidad, la alternativa es agregar un **barrido de video corto** por ambiente (10 s caminando junto a la pared). Queda detrás de una bandera de configuración y no cambia el resto del protocolo.

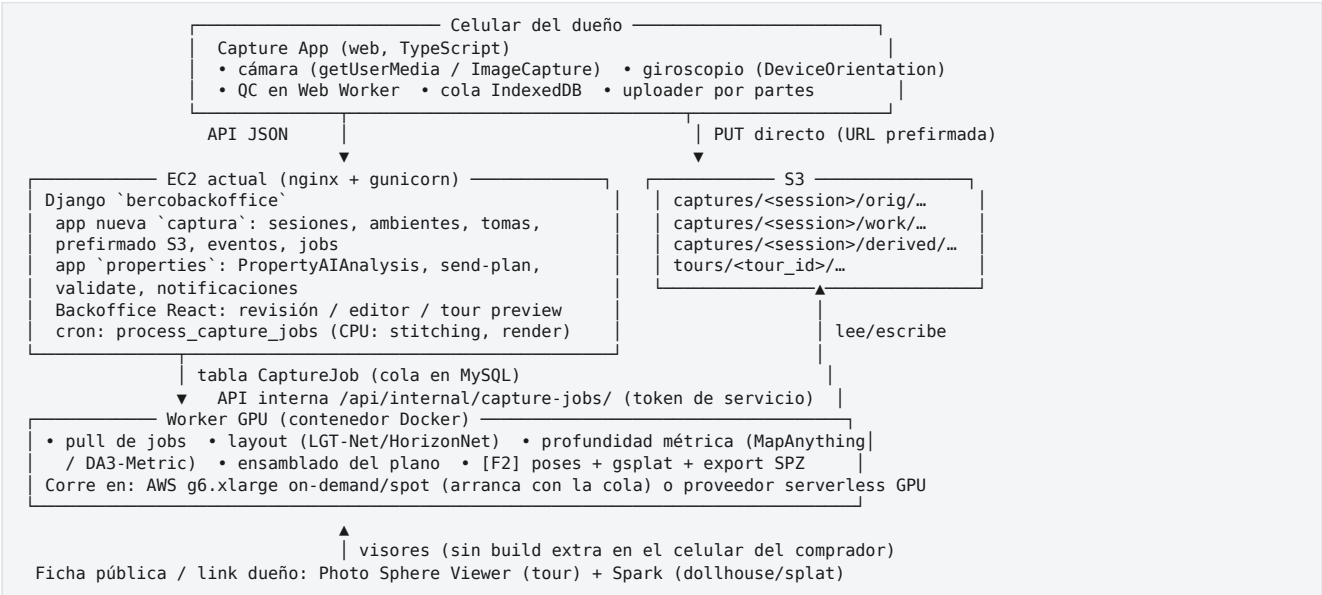
6.3 Metadatos que se guardan en cada toma (obligatorio)

```
{
  "shot_id": "uuid-v4 generado en el celular",
  "session_id": "...",
  "room_local_id": "uuid del ambiente (creado en el celular)",
  "kind": "pano",
  "index": 3,
  "sweep": 0,
  "captured_at": "2026-10-02T14:31:05.221-03:00",
  "orientation": {
    "quaternion": [x, y, z, w],
    "alpha": 134.2, "beta": 88.1, "gamma": -1.3,
    "absolute": false,
    "screen_angle": 0
  },
  "motion": { "rot_rate_dps": 4.1, "accel_rms": 0.12 },
  "camera": {
    "facing": "environment",
    "device_label": "Back Camera",
    "width": 1920, "height": 1440,
    "fov_h_deg_estimate": 66,
    "zoom": 1.0,
    "source": "getUserMedia|imagecapture|native_input"
  },
  "qc": { "blur_var": 212.4, "mean_luma": 118, "clipped_pct": 2.1, "tilt_ok": true, "passed": true, "attempt": 1 },
  "device": { "ua": "...", "platform": "iOS 26.0", "model_hint": "iPhone15,3", "dpr": 3 },
  "scale": { "type": "door_width", "value_m": 0.82 },
  "file": { "sha256": "...", "bytes": 1840231, "mime": "image/jpeg", "variant": "work|original" }
}
```

- **Pasos entre ambientes (opcional):** mientras el dueño camina de P9 a P5 se registra un log liviano del acelerómetro (conteo de pasos) y del yaw, para inicializar la distancia entre ambientes. No es crítico; ayuda si falla la alineación visual.
- **EXIF:** en el servidor se **elimina el GPS** del EXIF antes de publicar cualquier derivado. El original se conserva privado.

7. Arquitectura general

7.1 Diagrama



7.2 Componentes y responsabilidades

Componente	Tecnología	Dónde vive	Responsabilidad
Capture App	TypeScript + Vite, entrada separada (capture.html) en el repo bercobackoffice, sin React o con Preact para mantenerla < 150 kB gz	Servida como estáticos en re.bercovich.com/c/<token>	UX P0-P12, cámara, sensores, QC, cola local, subida
API de captura	Django app nueva backend/captura/	EC2 actual	Sesiones, ambientes, tomas, prefirmado S3, eventos, cola de jobs, API interna para el worker
Almacenamiento	S3 (bucket nuevo o prefijo en el existente, sección 12.2)	AWS	Originales privados; derivados públicos por CDN
Cola de jobs	Tabla CaptureJob en MySQL + cron / worker pull	EC2 + worker	Etapas del pipeline con reintentos e idempotencia
Jobs CPU	Python en EC2 (cron)	EC2	Miniaturas, QC de servidor, stitching 360 (OpenCV), render SVG/PNG/PDF
Worker GPU	Docker (PyTorch + CUDA)	GPU on-demand	Layout, profundidad métrica, ensamblado; F2: poses, splat
Backoffice	React existente	EC2	Diálogo de link, estado en vivo, revisión y editor de plano, preview del tour
Visores públicos	Photo Sphere Viewer 5 + plugins; Spark	Bundle dentro del sitio (CSP self)	Tour 360, dollhouse, splat

Por qué una entrada Vite separada y no el template Django actual: la captura es una aplicación de cámara con estado complejo (máquina de estados, workers, IndexedDB) que necesita tipado y tests. La entrada separada no carga el bundle del backoffice (pesa mucho en 4G) y reutiliza el toolchain del repo (Vite, Vitest, Playwright). El template actual owner_upload/page.html queda para el modo review legado hasta migrarlo.

7.3 Flujo de datos resumido

1. El backoffice hace POST /api/properties/{id}/capture-sessions/ y recibe la sesión + token + URL + texto de WhatsApp.
2. El dueño abre /c/<token>: el HTML estático hace GET /api/public/capture/<token> y recibe el estado de la sesión, la lista de ambientes y la config.
3. Por cada toma:

4. se guarda en IndexedDB;
5. POST `.../shots/` con metadatos, que devuelve URLs prefiradas;
6. PUT directo a S3;
7. POST `.../shots/{id}/complete/`.
8. POST `.../finish/` crea un `CaptureJob(stage=ingest)`. El pipeline avanza por etapas (sección 10) y al final crea o actualiza `PropertyAIAnalysis(status=draft)` y notifica.
9. Revisión → `send-plan` (existente) → el dueño valida (existente).
10. Fase 2: `CaptureJob(stage=tour_*)` sobre la misma sesión → `VirtualTour(status=ready)` → el asesor aprueba → se publica.

8. Captura en el celular: especificación técnica

8.1 Máquina de estados (cliente)

```
init → permissions → setup(P1) → rooms(P3) → entry(P4)
→ [por ambiente] room_select(P5) → scale?(P8, solo 1ra vez) → pano(P6) → corners(P7) → door(P9)
↑
→ review(P10) → finish(P11)
```

- Implementar con una máquina de estados explícita (XState o un reducer propio) **persistida en IndexedDB** en cada transición. Reabrir el link restaura el estado exacto.
- Cada ambiente tiene un `room_local_id` (UUID generado en el cliente). El servidor lo acepta tal cual: **idempotencia**.

8.2 Cámara

- `navigator.mediaDevices.getUserMedia({ video: { facingMode: { ideal: 'environment' }, width: { ideal: 4032 }, height: { ideal: 3024 }, aspectRatio: { ideal: 4/3 } }, audio: false })`.
- **Selección de lente:** enumerar `enumerateDevices()` y preferir la cámara trasera principal (1x). Probar en el piloto si la ultra gran angular (0.5x) está disponible y estable en iOS 26/Safari y Chrome Android. Si lo está, **reduce los objetivos del giro de 8 a 6** y mejora baños chicos. (**Verificar en dispositivos: no confirmado.**)
- **Captura del fotograma:**
- Chrome Android: `ImageCapture.takePhoto()` cuando esté disponible (resolución de foto completa, con enfoque y exposición automáticos).
- Safari iOS (sin `ImageCapture`): se dibuja el frame del `<video>` en un `OffscreenCanvas` o `canvas` y se exporta como `toBlob('image/jpeg', 0.92)`. La resolución queda limitada a la del stream (típicamente 1920×1440 o 1920×1080; **medir en el piloto**). Para el layout alcanza (los modelos trabajan a 512-1024 px).
- **Tamaño de pantalla / orientación:** la captura se hace en **vertical** (portrait). Bloquear la rotación de la UI con CSS (en web no se puede bloquear la orientación del sistema) y leer `screen.orientation.angle` para corregir la orientación.
- **Linterna:** no usarla (distorciona el color).
- **Estimación de FOV:** tabla por modelo de dispositivo (`model_hint`) + default de 66° horizontal en portrait. El servidor la refina durante el stitching (sección 10.2).

8.3 Sensores

- Fuente: `deviceorientation` (alpha/beta/gamma) → **cuaternión** (convertir con la convención W3C, ver `screen.orientation.angle`). En Android preferir `deviceorientationabsolute` si existe. En iOS, `webkitCompassHeading` es opcional y ruidoso: **no depender del norte absoluto**, solo del yaw relativo dentro de un giro.
- `devicemotion` para `rotationRate` (velocidad angular) y `accelerationIncludingGravity` (quietud y conteo de pasos).
- Frecuencia: la que dé el navegador (~60 Hz). Filtrar con promedio móvil de 5 muestras.
- **Deriva:** el yaw del giroscopio deriva unos grados por minuto. Se resetea la referencia al inicio de cada giro. El error residual se corrige en el stitching del servidor (los metadatos son solo inicialización).
- **Permiso iOS:** `DeviceOrientationEvent.requestPermission()` y `DeviceMotionEvent.requestPermission()` **dentro del mismo handler de click** que abre la cámara, con HTTPS. Si devuelve `denied` → modo asistido.

8.4 Almacenamiento local y variantes

- Cada toma genera:
- **work:** JPEG de lado mayor ≤ 2048 px, calidad 0.88 (~0.4-0.8 MB). Sube **primero**; es lo que el pipeline necesita.
- **original:** lo máximo que dio la cámara. Sube **después**, con menor prioridad, en segundo plano. Es opcional para el plano y útil para el tour y el splat en alta definición.
- Si el almacenamiento está justo (`estimate() < 300 MB` libres) → solo se guarda `work` y se marca `original_skipped=true`.
- Cola en IndexedDB: `{shot_id, blob_work, blob_original?, meta, state: pending|uploading|uploaded|failed, attempts}`.
- Se borran los blobs del celular solo cuando el servidor confirma `complete` (hash verificado).

8.5 Subida

- **Prefirmado S3 con subida directa desde el celular** (no pasa por gunicorn ni por nginx: se evita el límite de 50 MB y se libera el EC2).
- **work** (< 5 MB): PUT simple con URL prefirmada (validez 15 min).
- **original** (hasta 30 MB): **multipart upload S3** con partes de 5 MB; POST `.../multipart/` crea el upload y devuelve las URLs por parte, y al final se hace `complete`.
- **Concurrencia**: 2 subidas en paralelo en celular (3 en wifi, según `navigator.connection.effectiveType` donde exista).
- **Reintentos**: backoff exponencial 1 s, 3 s, 9 s, 27 s y luego cada 60 s mientras la página esté abierta. Si la URL prefirmada expiró (403), se pide una nueva.
- **Idempotencia**: `shot_id` generado en el cliente. POST `shots/` con un `shot_id` existente devuelve el mismo registro. El `complete` verifica `sha256` (el cliente lo calcula con `crypto.subtle.digest`) contra el ETag/tamaño de S3.
- **Limitación de iOS**: Safari no tiene Background Sync. Si el dueño cierra la pestaña, las subidas se pausan y **se reanudan al reabrir el link**. Mitigaciones: 1. subir durante la captura, no al final; 2. P10 no deja "Terminé" sin aviso si hay pendientes; 3. un recordatorio por WhatsApp desde el backoffice si la sesión quedó con pendientes más de 2 horas (sección 13).
- **Tolerancia de formatos**: siempre JPEG generado por la app. En el modo simple (<input>) pueden llegar HEIC o PNG; el servidor convierte (ya existe `pillow-heif`).

8.6 Control de calidad en el celular (Web Worker)

Por cada frame candidato, sobre una versión reducida (lado mayor 320 px, en escala de grises):

Chequeo	Método	Umbral inicial (ajustar en el piloto)	Mensaje
Nitidez	Varianza del Laplaciano	< 60 → rechazar	"Salió movida, quedate quieto un segundo"
Exposición	Luminancia media	< 45 → rechazar; 45-70 → avisar	"Está muy oscuro, prendé la luz"
Contraluz	% píxeles quemados (> 250)	> 25% → avisar	"Hay mucha luz de la ventana, probá correr la cortina"
Nivel	beta/gamma del sensor	pitch ±10°, roll ±5°	"Enderezá el celular"
Movimiento	<code>rotationRate</code>	> 15°/s	"Más despacio"
Obstrucción	Luminancia de un borde < 10% del resto		"Sacá el dedo de la cámara"

Los umbrales viven en la **config de la sesión**, que devuelve el servidor, para ajustarlos sin redeploy.

8.7 Modo simple (fallback)

- Se activa si falla `getUserMedia`, si el navegador no soporta lo necesario (Chrome < 100, Safari < 16), o si el dueño lo pide ("Prefiero usar la cámara de mi celular").
- Usa `<input type="file" accept="image/*" capture="environment">` por toma, con la **misma lista de tomas y los mismos textos**, e ilustraciones de hacia dónde apuntar. No hay disparo automático ni metadatos de orientación (se marca `source=native_input`).
- El pipeline funciona igual pero el stitching arranca sin poses iniciales (más lento, menos robusto). La confianza del plano baja un nivel.

8.8 Compatibilidad objetivo

Plataforma	Mínimo	Notas
iOS Safari	iOS 16.4+	Permiso de sensores con gesto; sin <code>ImageCapture</code> ; sin Background Sync
Chrome Android	110+	<code>ImageCapture</code> , <code>deviceorientationabsolute</code> , <code>vibrate</code>
Samsung Internet	20+	Probar
WebView de WhatsApp / Instagram	—	Detectar y pedir "Abrir en Safari/Chrome" (los in-app browsers suelen bloquear la cámara o los sensores)

9. Backend: modelos, API y carga

9.1 App nueva `backend/captura/`

Separada de `properties` para no seguir engordando ese módulo. Tiene FK a `properties.Property` y reutiliza `PropertyAIAnalysis`.

```

class CaptureSession(models.Model):
    id = UUIDField(primary_key=True)
    property = FK('properties.Property', related_name='capture_sessions')
    upload_link = OneToOneField('properties.PropertyUploadLink', null=True) # reutiliza token/expiration/revocación
    status = CharField(choices=[created, opened, in_progress, finished, processing,
                               plan_draft, plan_approved, plan_validated, failed, abandoned])
    owner_inputs = JSONField(default=dict) # ambientes, m² declarados, balcón, piso/depto
    device_info = JSONField(default=dict)
    capture_mode = CharField(choices=[guided, assisted, simple])
    scale_ref = JSONField(null=True) # {type:'a4'|'door_width'|'none', value_m, shot_id}
    protocol_version = CharField(default='1.0')
    config_snapshot = JSONField(default=dict) # umbrales de QC y flags usados
    opened_at, started_at, finished_at = DateTimeField(null=True)
    created_by = FK(User)
    created_at / updated_at

class CaptureRoom(models.Model):
    id = UUIDField(primary_key=True) # = room_local_id del cliente
    session = FK(CaptureSession, related_name='rooms')
    order = PositiveSmallIntegerField()
    room_type = CharField() # living, cocina, dormitorio, baño, lavadero, balcon, pasillo, otro
    label = CharField() # "Dormitorio 2"
    is_large = BooleanField(default=False)
    status = CharField(choices=[pending, partial, complete, redo_requested])

class CaptureShot(models.Model):
    id = UUIDField(primary_key=True) # = shot_id del cliente (idempotencia)
    session = FK(CaptureSession, related_name='shots')
    room = FK(CaptureRoom, null=True)
    kind = CharField() # entry_in, entry_door, pano, pano_up, corner, door_from, door_to, scale_top,
    scale_ctx, scale_door, extra
    index = SmallIntegerField(default=0)
    sweep = SmallIntegerField(default=0)
    target_room = FK(CaptureRoom, null=True, related_name='+') # para door_*
    meta = JSONField() # sección 6.3 completa
    work_key = CharField() # S3 key
    original_key = CharField(blank=True)
    sha256_work = CharField(); bytes_work = IntegerField()
    upload_state = CharField(choices=[pending, work_uploaded, complete])
    qc_server = JSONField(null=True)
    captured_at = DateTimeField()
    created_at

class CaptureJob(models.Model):
    session = FK(CaptureSession, related_name='jobs')
    stage = CharField() # ingest, qc, stitch, layout, scale, assemble, render, tour_nodes, dollhouse, splat_poses,
    splat_train, splat_export
    runner = CharField(choices=[cpu, gpu])
    status = CharField(choices=[queued, running, done, failed, skipped])
    attempts = SmallIntegerField(default=0)
    locked_by = CharField(blank=True); locked_until = DateTimeField(null=True)
    input_version = CharField() # hash de shots + protocol + model versions → idempotencia/reproceso
    output = JSONField(null=True); error = TextField(blank=True)
    timings = JSONField(default=dict); cost_usd = DecimalField(null=True)
    created_at / updated_at

class CaptureEvent(models.Model):
    # telemetría server-side (sección 13)
    session = FK(CaptureSession, null=True)
    name = CharField(db_index=True) # link_open, permissions_ok, room_start, shot_ok, shot_retry, ...
    screen = CharField(blank=True) # P0..P12
    payload = JSONField(default=dict)
    client_ts = DateTimeField(null=True); created_at = DateTimeField(auto_now_add=True, db_index=True)

class VirtualTour(models.Model):
    # Fase 2
    session = FK(CaptureSession); property = FK('properties.Property')
    status = CharField(choices=[building, ready, approved, published, failed])
    nodes = JSONField() # [{room_id, pano_key, position_xy, heading, links:...}]
    dollhouse_key = CharField(blank=True); splat_key = CharField(blank=True)
    public_slug = SlugField(unique=True, null=True)
    version = PositiveIntegerField()

```

Plano (resultado de Fase 1): se guarda en `PropertyAIAnalysis.rooms` con un **nuevo esquema versionado** (`schema: "plan/2"`), sin romper el actual (`schema` ausente = legado):

```
{
  "schema": "plan/2",
  "units": "m",
  "rooms": [{
    "id": "uuid", "room_type": "dormitorio", "label": "Dormitorio 1",
    "polygon": [[0,0],[3.42,0],[3.42,2.95],[0,2.95]],
    "width_m": 3.42, "length_m": 2.95, "area_m2": 10.09, "ceiling_h_m": 2.62,
    "confidence": 0.82, "source": "layout_pano+corners+scale_a4",
    "doors": [{ "to": "uuid-living", "segment": [[1.1,2.95],[1.9,2.95]]}],
    "windows": [{ "segment": [[3.42,0.8],[3.42,2.0]]}],
    "shots": ["uuid", "..."]
  }],
  "adjacencies": [{ "a": "uuid", "b": "uuid", "kind": "door"}],
  "totals": { "util_m2": 58.4, "cubierta_est_m2": 64.9, "wall_thickness_assumed_m": 0.15, "semicubierta_m2": 4.2},
  "scale": { "method": "a4", "residual_pct": 1.8},
  "quality": { "overall_confidence": 0.78, "warnings": ["baño: solo 2 esquinas"]},
  "models": { "layout": "lgt-net@zind-v1", "depth": "mapanything-apache@1.1", "stitch": "opencv-4.10"}
}
```

Frontend: actualizar `src/types/owner-upload.ts` con este tipo (`PlanV2`) y un adaptador para el legado. De paso queda arreglado el bug de contrato (sección 2.2, punto 7).

9.2 API

Pública (token en la URL; host `re.bercovich.com`; rate limit por token+IP; CORS solo ese host):

Método	Ruta	Cuerpo / respuesta
GET	<code>/api/public/capture/<token>/</code>	→ {session, property:{address}, owner_name, rooms[], shots_summary, config:{qc_thresholds, flags}, mode}; registra <code>link_open</code>
POST	<code>/api/public/capture/<token>/inputs/</code>	{rooms_count, m2_declared, has_balcony, floor_unit} → sesión
PUT	<code>/api/public/capture/<token>/rooms/<room_id>/</code>	upsert idempotente {room_type, label, order, is_large}
POST	<code>/api/public/capture/<token>/shots/</code>	{shot_id, room_id, kind, index, meta, sha256, bytes, has_original} → {work_put_url, original: {upload_id?}} (idempotente)
POST	<code>/api/public/capture/<token>/shots/<shot_id>/original/parts/</code>	{part_numbers:[...]} → URLs prefiradas por parte
POST	<code>/api/public/capture/<token>/shots/<shot_id>/complete/</code>	{variant:'work' 'original', etag?, parts?} → verifica en S3 (HEAD) y marca estado
POST	<code>/api/public/capture/<token>/events/</code>	batch [{name, screen, payload, client_ts}] (máx. 50)
POST	<code>/api/public/capture/<token>/finish/</code>	→ crea <code>CaptureJob(ingest)</code> ; debounce : si ya hay un job activo, devuelve el existente
GET	<code>/api/public/capture/<token>/preview/</code>	→ estado del 360 en vivo del primer ambiente (P11)
—	<code>validate/, valuation/</code>	se reutilizan los actuales de <code>views_owner_public.py</code>

Interna (backoffice, isAuthenticated, camelCase como el resto):

Método	Ruta	Uso
POST/GET	<code>/api/properties/{id}/capture-sessions/</code>	Crear sesión (crea también el <code>PropertyUploadLink</code>) / listar
GET	<code>/api/capture-sessions/{id}/</code>	Estado en vivo, ambientes, tomas, jobs, eventos resumidos
POST	<code>/api/capture-sessions/{id}/rooms/{room_id}/request-redo/</code>	Genera el deep-link para rehacer un ambiente
POST	<code>/api/capture-sessions/{id}/reprocess/</code>	Reencola desde una etapa ({from_stage})
PATCH	<code>/api/properties/{id}/ai-analysis/</code>	Extender : aceptar <code>plan/2</code> , editar cotas → re-render del SVG
GET/POST	<code>/api/capture-sessions/{id}/tour/</code>	Fase 2: estado, aprobar, publicar

Worker (servicio a servicio, header `Authorization: Bearer <CAPTURE_WORKER_TOKEN>`, IP allowlist opcional):

Método	Ruta	Uso
POST	<code>/api/internal/capture-jobs/claim/</code>	{runner:'gpu', stages:[...]} → job + URLs prefiradas de lectura/escritura; lock de 30 min
POST	<code>/api/internal/capture-jobs/{id}/heartbeat/</code>	Extiende el lock
POST	<code>/api/internal/capture-jobs/{id}/result/</code>	{status, output, timings, cost_usd} → la cola avanza a la siguiente etapa

9.3 Orquestación de etapas

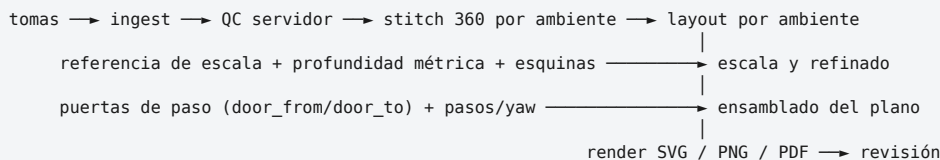
- El **cron existente en el EC2** (patrón `deploy/cron-*`) corre `manage.py process_capture_jobs` cada minuto con `flock`:
- toma `jobs runner=cpu` (`ingest`, `qc`, `stitch`, `render`);
- cuando un job termina, crea el siguiente de la cadena;
- para `jobs runner=gpu`, si no hay worker vivo, **arranca la instancia GPU** (sección 12.3).
- Cadena Fase 1: `ingest` → `qc` → `stitch` → `layout(gpu)` → `scale(gpu)` → `assemble(gpu)` → `render(cpu)`.
- Cadena Fase 2: `tour_nodes(cpu)` → `dollhouse(cpu)` → [`splat_poses(gpu)` → `splat_train(gpu)` → `splat_export(cpu)`] (entre corchetes, opcional por flag).
- **Idempotencia**: `input_version` = `hash(ids+sha de las tomas usadas, protocol_version, versiones de modelos)`. Si ya existe un job `done` con el mismo `input_version`, se salta.
- **Reproceso masivo**: `manage.py reprocess_captures --stage layout --since 2026-10-01` al actualizar un modelo. Así se regeneran los planos o se generan los tours de la Fase 2 sobre capturas viejas.

9.4 Corrección del job actual

Además de lo nuevo, en la Fase 0 hay que agregar `deploy/cron-process-owner-analyses` para el pipeline legado mientras conviva.

10. Pipeline Fase 1: del set de fotos al plano 2D

10.1 Visión general



10.2 Paso a paso

1. Ingest (CPU) - Verifica que cada ambiente tenga lo mínimo: $\geq 6/8$ del giro y ≥ 1 esquina, o bien ≥ 1 par de puertas. - Genera miniaturas con `image_pipeline.generate_variants` (reutilizar). - Elimina el GPS del EXIF en derivados. - Estado de la sesión → `processing`.

2. QC de servidor (CPU + LLM existente) - Repite las métricas del cliente con más resolución. - **Reutiliza el stage 0/1 de owner_ai_pipeline.py** (gate de calidad y tipo de ambiente) para: - confirmar el `room_type` elegido por el dueño; - detectar tomas que no son de la propiedad; - generar la **evaluación de estado** (impeccable / a refaccionar) que ya existe. - El resultado queda en `CaptureShot.qc_server` y alimenta la confianza.

3. Stitching 360 por ambiente (CPU) - Entrada: 8 tomas `pano_k` (+ `pano_up_k`) con cuaterniones. - OpenCV `Stitcher` en modo PANORAMA con warper esférico: - rotaciones iniciales tomadas de los cuaterniones (reduce fallos en paredes lisas); - bundle adjustment de rotación y focal (esto refina el FOV). - Salida: equirectangular 4096×2048 (JPEG) + máscara de huecos (cenit/nadir faltantes, se rellenan con inpainting simple o se dejan en gris neutro). - **Fallback**: si el stitching falla (pocas features), se proyectan las tomas con las rotaciones del sensor sin optimizar. Resultado usable para el layout, con confianza menor. - **En vivo (P11)**: el stitching del primer ambiente se encola apenas terminan sus 8 tomas, para mostrar el 360 al final de la captura. Es un job CPU de ~5-15 s.

4. Layout por ambiente (GPU) - Modelo **LGT-Net** con pesos de ZInD (casas reales) y **HorizonNet** como segunda opinión. Ambos MIT. - Entrada: equirectangular alineada a la vertical (usar la gravedad del sensor para la corrección de horizonte; HorizonNet trae preprocesamiento de alineación Manhattan). - Salida: polígono del piso en coordenadas de cámara, escala relativa a la altura de cámara, y altura de techo relativa. - **Ensamble**: si LGT-Net y HorizonNet discrepan > 10% en área, se baja la confianza y el ambiente se marca para revisión. - Detección de puertas y ventanas en el panorama: HorizonNet/LGT-Net no las dan. Primera versión: ubicar puertas solo a partir de las tomas `door_*`. Las ventanas se detectan con un clasificador ligero o se omiten en la v1 (decisión abierta, sección 18).

5. Escala y refinado (GPU) - Problema: el layout desde un panorama da la forma con escala relativa a la altura de la cámara. Un error de ± 5 cm en la altura (sobre ~1,40 m) da ~3-4% de error lineal y ~6-7% de área. Hay que fijar la escala con datos. - **Fuentes de escala, en orden de prioridad**: 1. **Hoja A4** (`scale_top`): detección del rectángulo (contornos + homografía, 210×297 mm) → escala absoluta del piso en la toma → se transfiere al panorama por la pose relativa (la hoja también está en `scale_ctx` y en el giro). 2. **Puerta medida** (`scale_door` + valor): se detecta el marco en la toma y se asigna el ancho. 3. **Profundidad métrica multi-vista**: **MapAnything** (pesos `map-anything-apache`) sobre giro + esquinas del ambiente, que da poses y profundidad métrica. Se ajustan los planos de pared a la nube y se estima la altura real de la cámara. Alternativas: **Depth Anything 3 METRIC-LARGE** (Apache) o **MoGe-2** (MIT) monocular por toma. 4. **Priors**: puerta interior estándar de 2,00-2,05 m de alto, altura de techo típica de CABA de 2,50-2,80 m. Solo como validación, nunca como fuente principal. 5. **m² declarados** por el dueño: solo como alerta (si el total difiere > 15%, se marca la advertencia para el revisor). **No se re-escala**

automáticamente, a diferencia del pipeline actual (`_scale_areas_to_total`), que esconde los errores. - **Refinado con esquinas:** se proyectan las esquinas del polígono del layout en las tomas `corner_k` con las poses de MapAnything, se ajustan los vértices por mínimos cuadrados (rectificado de ángulos a 90° cuando el ambiente es Manhattan con $\pm 3^\circ$) y se registra el residuo como métrica de confianza. - Salida por ambiente: polígono en metros, altura de techo, confianza.

6. Ensamblado del plano (GPU/CPU) - Grafo de ambientes: nodos = ambientes; aristas = pares `door_from/door_to`. - **Alineación:** para cada puerta, se ubica el vano en el polígono de ambos ambientes (detección del marco en las tomas de puerta + correspondencia con el panorama por pose) y se calcula la transformación rígida 2D que hace coincidir los vanos, con las paredes compartidas en paralelo. Optimización global (grafo de poses 2D, por ejemplo con `scipy.optimize.least_squares`) minimizando: - la distancia entre vanos correspondientes; - la superposición entre ambientes; - el desvío respecto del espesor de muro asumido (0,10-0,20 m). - Inicialización: yaw del sensor (relativo al giro de entrada) + conteo de pasos. - **Fallback:** si un ambiente no queda conectado (faltan tomas de puerta), se ubica al costado con la etiqueta "ubicación aproximada" y se avisa al revisor. - **Totales:** - superficie **útil** = suma de polígonos interiores; - **cubierta estimada** = contorno exterior con el espesor de muro asumido (0,15 m interiores; 0,30 m medianeras/exterior, configurable); - **semicubierta** = balcones. - Se rotulan explícitamente como estimaciones.

7. Render (CPU) - Evolucionar `floorplan_render.py` a un renderer de polígonos en metros: - escala gráfica; - cotas por pared (ancho x largo) y m² por ambiente; - puertas con arco de apertura, ventanas; - norte opcional; - leyenda "Plano informativo — medidas estimadas a partir de fotos, tolerancia $\pm 5\%$ "; - logo. - Salidas: - SVG para la web; - PNG 2400 px (instalar **cairosvg** y dejar Pillow como fallback); - **PDF A4** (cairosvg o reportlab) para "Descargar plano". - Se guarda como hoy: `PropertyAIAnalysis(version=n+1, status=draft, rooms=plan/2, floorplan_svg=...)` + `PropertyImage(source='owner', alt='Plano generado')`. **Excluir ese alt del input de cualquier pipeline** (bug 2.2.8). - Notificar `OWNER_ANALYSIS_DRAFT` al asesor.

10.3 Confianza y reglas de revisión

- `confidence` por ambiente = $f(\text{QC de tomas, acuerdo LGT/Horizon, residuo de esquinas, método de escala})$.
- Umbrales:
- $\geq 0,8$ verde;
- **0,6-0,8** amarillo (el revisor mira las cotas);
- $< 0,6$ rojo (el revisor corrige o pide rehacer el ambiente).
- La sesión no puede aprobarse con ambientes en rojo sin editar o justificar (por ejemplo, un checkbox "revisado manualmente").

10.4 Tiempos y costos estimados por propiedad (a medir en el piloto)

Etap	Dónde	Tiempo	Costo
Ingest + QC + LLM	CPU + API	1-3 min	~USD 0,02-0,05 (tokens, como hoy)
Stitch (7 ambientes)	CPU EC2	1-2 min	~0
Layout + escala + ensamblado	GPU (L4 24 GB)	2-5 min	~USD 0,05-0,15 (a ~USD 0,8-1/h on-demand)
Render	CPU	< 30 s	~0
Total F1		5-15 min	< USD 0,30

11. Pipeline Fase 2: recorrido virtual 3D

Se construye **sobre las mismas tomas y el plano ya calculado**; no hay captura nueva.

11.1 Nivel 1: Tour 360 (primer entregable de F2)

- **Nodos:** un nodo por giro (`sweep`). Posición = centro del giro en el plano ensamblado; heading = yaw del giro alineado al plano.
- **Enlaces:** por cada adyacencia de puerta se crea un enlace bidireccional entre los nodos de ambos ambientes, con el marcador ubicado en la dirección del vano (calculada desde la pose).
- **Visor: Photo Sphere Viewer 5** + plugins `virtual-tour` (nodos y enlaces), `gallery`, `map` (plano como minimapa con posición y dirección de vista) y `markers`. Todo MIT y **bundleado** (la CSP es `script-src 'self'`; nada por CDN externo).
- **Imágenes:** equirectangulares a 4096x2048 con tiles (plugin `equirectangular-tiles`) para carga rápida en 4G. Versión 2048 para preview.
- **Entregables:** embed en la ficha pública, en el link del dueño y en la URL `re.bercovich.com/t/<slug>` con OG image (panorama del living).

11.2 Nivel 2: Dollhouse

- Extruir cada polígono con su altura de techo; texturizar paredes y piso proyectando el panorama del ambiente desde su centro (proyección esférica inversa). Se generan techos abiertos para la vista de maqueta.
- Formato: glTF/GLB (`trimesh` en Python) de ~5-15 MB.

- Visor: three.js (dentro del bundle) con transición "vista maqueta ↔ entrar al nodo 360", como hace Matterport.
- Sin GPU; es geometría + reproyección.

11.3 Nivel 3: Reconstrucción fotorrealista (Gaussian splatting): opcional, detrás de flag

- **Poses:** MapAnything (Apache) sobre todas las tomas (giro + esquinas + puertas) con las poses de sensor como prior. Alternativa: **VGGT-1B-Commercial** (licencia comercial con formulario) o COLMAP (BSD) como fallback clásico.
- **Entrenamiento:** **gsplat** (Apache-2.0), 7–15k iteraciones, en GPU L4/A10G. Unos 15–40 min por propiedad; **medir**.
- **Export:** comprimido SPZ o SOG (~10–40 MB) y visor **Spark** (MIT, THREE.js, apto para celulares de gama media).
- **Criterio de publicación:** métricas automáticas (PSNR en tomas retenidas $\geq$ umbral), revisión visual del asesor y, si no pasa, se publica solo el Nivel 1 + 2.
- Si el piloto muestra que las esquinas y las puertas no dan suficiente paralaje, se activa el **barrido de video** por ambiente (flag `capture.video_sweep`, sección 6.2).

11.4 Costo estimado F2

- Nivel 1 + 2: < USD 0,05 (CPU).
- Nivel 3: USD 0,5–2 de GPU + almacenamiento y transferencia del splat. **A medir**.

12. Infraestructura y puesta en producción

12.1 Cambios en nginx (`deploy/nginx-bercobackoffice.conf`)

Crear un **server separado para** `re.bercovich.com` (hoy hay un único `default_server`). Así los headers de la captura no afectan al backoffice:

```
server {
    server_name re.bercovich.com;
    # TLS (certbot) igual que el server actual

    # Cámara y sensores S0L0 en este host
    add_header Permissions-Policy "camera=(self), accelerometer=(self), gyroscope=(self), magnetometer=(self), microphone=()"
    always;
    add_header Content-Security-Policy "default-src 'self'; img-src 'self' data: blob: https://<bucket-cdn>; media-src 'self' blob;; connect-src 'self' https://<bucket>.s3.<region>.amazonaws.com; worker-src 'self' blob;; script-src 'self'; style-src 'self' 'unsafe-inline'; frame-ancestors 'self' https://bercovich.com https://www.bercovich.com;" always;

    location /c/ { root /opt/bercobackoffice/capture; try_files /index.html =404; } # SPA de captura
    location /capture-assets/ { root /opt/bercobackoffice/capture; expires 1y; add_header Cache-Control "public, immutable"; }
    location /t/ { root /opt/bercobackoffice/capture; try_files /tour.html =404; } # visor público (F2)
    location /p/ { proxy_pass http://127.0.0.1:8000; include proxy_params; } # legado (review) – hoy falta
    location /api/public/ { proxy_pass http://127.0.0.1:8000; include proxy_params; client_max_body_size 5M; }
}
```

- En el server del backoffice **no** se habilita la cámara (queda `camera=()`).
- `client_max_body_size` del backoffice: subir a 210M si el legado sigue aceptando videos de 200 MB (bug 2.2.3), o bajar el límite de la app a 50 MB.

12.2 S3

- **Bucket:** si `AWS_STORAGE_BUCKET_NAME` ya está activo en prod, usar un **prefijo** `captures/` con política separada. Si no, crear `bercovich-captures` (privado).
- `captures/<session>/orig/...` y `.../work/...`: **privado**, solo accesible por URLs prefirmadas.
- `captures/<session>/derived/...` y `tours/...`: público vía CloudFront (o el `AWS_S3_CUSTOM_DOMAIN` existente) con `Cache-Control: immutable`.
- **CORS del bucket** (necesario para el PUT desde el navegador):

```
[{"AllowedOrigins": ["https://re.bercovich.com"], "AllowedMethods": ["PUT", "GET", "HEAD"],
  "AllowedHeaders": ["*"], "ExposeHeaders": ["ETag"], "MaxAgeSeconds": 3000}]
```

- **Lifecycle:**
- abortar multipart incompletos a los 7 días;
- `orig/` a Glacier IR a los 180 días;
- sesiones abandonadas (sin `finish`) se borran a los 60 días.
- **IAM:**
- usuario/rol del EC2 con `s3:PutObject/GetObject/HeadObject/AbortMultipartUpload/ListMultipartUploadParts` sobre `captures/*` y `tours/*`;
- rol del worker GPU con lectura en `captures/*` y escritura en `captures/*/derived/*` y `tours/*`.

- Dependencia: agregar `django-storages[s3]` y `boto3` a `requirements.txt` si no están (hoy el setting lo menciona pero el paquete no figura en `requirements.txt`; **verificar en el EC2**).

12.3 Worker GPU

- **Imagen Docker** `bercovich/capture-worker` (repo nuevo `capture-worker`, o carpeta `workers/capture/` en `bercobackoffice`):
- base `pytorch/pytorch:2.x-cuda12.x-runtime`;
- `opencv-python-headless`, `numpy`, `scipy`, `trimesh`, `shapely`, LGT-Net y HorizonNet (vendorizados con licencia MIT), `map-anything`, `depth-anything-3` (solo pesos Apache), `gsplat` (F2);
- pesos descargados en el build, desde Hugging Face con hash fijado.
- **Loop del worker:** `claim` → descargar tomas por URL prefirmada → procesar → subir derivados → `result`. Heartbeat cada 60 s. Si no hay jobs durante 10 min, **se apaga** (la instancia hace `shutdown -h now`).
- **Opción A (recomendada para arrancar): AWS EC2 g6.xlarge** (NVIDIA L4 24 GB) en la misma región que el bucket:
- el cron de la sección 9.3 hace `ec2:StartInstances` cuando hay jobs GPU en cola y no hay worker vivo;
- arranque en frío de ~2-3 min;
- costo on-demand ≈ USD 0,8/h (**verificar el precio vigente en la región**); con ~10 propiedades por día, menos de 1 h de GPU diaria;
- spot para Fase 2 (splats) si se tolera la interrupción (el job se reintenta).
- **Opción B:** proveedor serverless GPU (Modal, RunPod, Replicate). Menos ops, se paga por segundo, pero los datos salen de AWS. Evaluar si el volumen crece o si el arranque en frío de la A molesta.
- Variables del worker: `CAPTURE_API_BASE`, `CAPTURE_WORKER_TOKEN`, `WORKER_STAGES=layout,scale,assemble[,splat_*]`, `MODEL_CACHE_DIR`, `IDLE_SHUTDOWN_MIN=10`.

12.4 Variables de entorno nuevas en el backend (/opt/bercobackoffice/backend/.env)

Variable	Ejemplo	Uso
<code>CAPTURE_PUBLIC_HOST</code>	<code>re.bercovich.com</code>	Host de la captura (puede reutilizar <code>PUBLIC_OWNER_UPLOAD_HOST</code>)
<code>CAPTURE_S3_BUCKET</code>	<code>bercovich-captures</code>	Si es distinto del bucket general
<code>CAPTURE_S3_REGION</code>	<code>us-east-1</code>	
<code>CAPTURE_CDN_BASE</code>	<code>https://cdn.bercovich.com</code>	Derivados públicos
<code>CAPTURE_WORKER_TOKEN</code>	(secreto)	Autenticación del worker
<code>CAPTURE_GPU_INSTANCE_ID</code>	<code>i-0abc...</code>	Para el arranque automático
<code>CAPTURE_QC_CONFIG</code>	JSON	Umbral de QC por defecto
<code>CAPTURE_FLAGS</code>	<code>{"pano_up":true,"video_sweep":false,"splat":false}</code>	Flags de protocolo
<code>CAPTURE_WHATSAPP_REMINDER_HOURS</code>	2	Recordatorio de subidas pendientes

12.5 Build y deploy

- **Vite multi-page:** agregar `capture.html` y `tour.html` en `vite.config.ts` (`build.rollupOptions.input`) con base `/capture-assets/`. Script `npm run build:capture` que publica en `/opt/bercobackoffice/capture`.
- Backend: migraciones de la app `captura`, `INSTALLED_APPS`, URLs en `backend/config/urls.py`.
- Crons nuevos en `deploy/`:
- `cron-process-capture-jobs` (cada minuto, con `flock`);
- `cron-capture-reminders` (cada 30 min);
- `cron-process-owner-analyses` (legado, Fase 0).
- Actualizar `deploy/RUNBOOK_CRM_PROD.md` o crear `deploy/RUNBOOK_CAPTURA.md` con:
- pasos de deploy;
- cómo arrancar y parar el worker;
- cómo reprocesar;
- cómo rotar el token del worker.
- **Feature flag por usuario/rol:** el botón nuevo del backoffice se habilita primero para los asesores del piloto.

13. Observabilidad, logs y analítica

Hoy no hay forma de saber si los dueños abren el link. Desde el día uno, **todo se registra en el servidor** (no depende de GA4 ni de bloqueadores):

13.1 Eventos (CaptureEvent)

Evento	Pantalla	Payload clave
link_open	P0	ua, referrer (¿in-app de WhatsApp?), primera vez o retorno
start	P0	
inputs_saved	P1	ambientes, m² declarados
permissions_result	P2	cámara ok/fail, sensores granted/denied, modo resultante, resolución real
room_start / room_complete	P5-P9	room_type, duración, tomas, reintentos
shot_ok / shot_retry	P4-P9	kind, motivo del retry (blur, dark, tilt, motion)
scale_done	P8	método
upload_error	—	código, intento, tipo de red
review_view / finish	P10-P11	pendientes de subida
abandon_suspected	—	el servidor lo marca si no hay eventos durante 30 min con la sesión en curso
plan_view / plan_validated / plan_feedback / valuation_request	P12	
tour_view	Tour	origen (ficha, link dueño, portal)

- **Batch:** el cliente acumula y manda cada 10 s o al cambiar de pantalla (`navigator.sendBeacon` al ocultarse la página).
- **Tablero en el backoffice** (sección "Marketing" o "Captación"): embudo por paso, tiempos, motivos de reintento, distribución de dispositivos, errores de subida, costo por propiedad.
- GA4 opcional: si se define el ID, reenviar los mismos nombres de evento.

13.2 Logs y alertas

- Logger captura con `session_id` en cada línea. Errores de jobs → notificación `CAPTURE_JOB_FAILED` al responsable técnico (Nico).
- Alertas:
- job en `running` más de 30 min;
- cola GPU con jobs de más de 20 min de espera;
- tasa de `upload_error` > 5% por hora;
- más de 3 fallos seguidos de la misma etapa.
- Comando `manage.py capture_stats --days 7` para diagnóstico rápido por consola.

13.3 Recordatorios automáticos

- Sesión `opened` sin `start` durante 24 h → aviso al asesor ("el dueño abrió el link pero no empezó").
- Sesión `in_progress` sin actividad durante 2 h y con subidas pendientes → aviso al asesor con el texto sugerido para WhatsApp: "Te quedaron fotos por subir, abrí el link de nuevo con wifi y se suben solas".

14. Seguridad, privacidad y licencias

14.1 Seguridad

- **Token:**
- reutilizar el firmado actual (`upload_links.py`), revocable, con expiración;
- el token nunca da acceso a otra propiedad;
- las URLs prefiradas se emiten por toma, con prefijo fijado a la sesión y 15 min de validez.
- **Rate limit:** reutilizar `owner_upload_guards` (30 req/min por token+IP), ampliado para la captura (~120 req/min, porque hay más tomas y eventos). Máximo 250 tomas y 3 GB por sesión.
- **Validación en servidor:** en `complete/`, HEAD al objeto para verificar el tamaño y el tipo real (bytes mágicos de la primera parte con `Range`).
- **Worker:** token de servicio rotante. Solo recibe URLs prefiradas, sin credenciales generales de S3.
- **CSP y Permissions-Policy** estrictas y por host (sección 12.1).

14.2 Privacidad

- Texto de consentimiento en P0: "Las fotos se usan para armar el plano y el recorrido de tu propiedad. Se publican solo si vos y tu asesor lo aprueban."

- **Personas en las fotos:** los textos piden que no haya nadie en el ambiente. En Fase 2, antes de publicar, **difuminar rostros** con un detector liviano en el pipeline y hacer revisión humana.
- EXIF GPS eliminado en derivados; originales privados.
- Retención:
- sesiones abandonadas: 60 días;
- capturas de propiedades dadas de baja: 1 año y luego se archivan;
- el dueño puede pedir el borrado (endpoint interno para el asesor).

14.3 Licencias (solo uso comercial permitido)

Componente	Licencia	¿Se puede usar?
LGT-Net, HorizonNet	MIT	Sí
ZInD (pesos entrenados con el dataset)	Apache-2.0 el código; el dataset tiene términos propios	Verificar los términos de uso de pesos derivados antes de producción
MapAnything (map-anything-apache)	Apache-2.0	Sí
Depth Anything 3 SMALL/BASE/METRIC-LARGE/MONO-LARGE	Apache-2.0	Sí. No las variantes GIANT/LARGE/NESTED (CC-BY-NC)
MoGe	MIT	Sí
VGGT-1B-Commercial	Licencia comercial (formulario)	Sí, con aprobación
gsplat	Apache-2.0	Sí
Photo Sphere Viewer, Spark, three.js	MIT	Sí
OpenCV, COLMAP	Apache-2.0 / BSD	Sí
DUST3R, MAST3R, MAST3R-SLAM, Pi3, SpatialLM (encoders), InstantSplat, 3DGS original de Inria	No comerciales	No usar
OpenSplat	AGPLv3	Evitar (copyleft)

Mantener `workers/capture/LICENSES.md` con versión, fuente y hash de cada peso.

15. Plan de pruebas y criterios de aceptación

15.1 Pruebas automáticas

- **Backend (pytest, patrón de `backend/properties/test_*.py`):**
- idempotencia de `shots/` y `rooms/`;
- prefirmando y `complete` con S3 simulado (moto);
- límites y rate limit;
- máquina de estados de la sesión;
- cadena de jobs y `input_version`;
- debounce de `finish/`;
- migración del esquema `plan/2` y el adaptador legado;
- exclusión de `alt='Plano generado'`.
- **Pipeline:** set de regresión ("**golden set**") con 10 capturas reales anotadas (plano medido con láser). En cada cambio de modelo o código, CI reporta el error de área por ambiente y el total; falla si empeora > 1 punto porcentual.
- **Frontend (Vitest):**
- máquina de estados;
- conversión de orientación a cuaternión;
- lógica de disparo automático (con sensores simulados);
- QC en el Worker (imágenes de prueba borrosas u oscuras);
- cola IndexedDB (`fake-indexeddb`) con cortes de red simulados.
- **E2E (Playwright, ya configurado en el repo):**
- flujo P0→P11 con cámara falsa (`--use-fake-device-for-media-stream` + video de prueba) y eventos de orientación inyectados;
- corte de red a mitad de camino;
- recarga de la página y reanudación;
- link vencido.

15.2 Matriz de dispositivos (manual, antes del piloto)

Dispositivo	SO / navegador	Qué validar
iPhone 12-13	iOS 17 / Safari	Permisos, resolución real del stream, disparo automático, reanudación
iPhone 15-16	iOS 26 / Safari	Idem + disponibilidad del ultra gran angular

Dispositivo	SO / navegador	Qué validar
Samsung gama media (A34/A54)	Android 14 / Chrome	ImageCapture, rendimiento del QC
Motorola gama baja (G-series)	Android 13 / Chrome	Rendimiento, memoria, almacenamiento
Xiaomi Redmi	Android / Chrome	Sensores (hay modelos sin giroscopio real)
Cualquiera	Link abierto desde WhatsApp (in-app)	Detección y "Abrir en el navegador"

15.3 Piloto de campo (criterio de salida de la Fase 1)

- **Muestra:** 10 propiedades reales (mezcla: 2 monoambientes, 4 de 2-3 ambientes, 2 PH, 2 de 4+ ambientes, al menos 2 con baños chicos y 2 con ventanales grandes).
- **Protocolo:** 1. Nico (o un asesor) mide cada ambiente con **medidor láser** (ancho y largo, más la altura de techo) y hace un croquis de referencia. 2. **El dueño hace la captura solo**, sin ayuda; se observa sin intervenir y se anotan dudas y trabas. 3. Se procesa y se compara.
- **Métricas a reportar:**
 - error de área por ambiente (mediana y p90);
 - error de área total útil;
 - error de ancho/largo en cm;
 - tiempo de captura;
 - tomas repetidas;
 - abandono;
 - escala de satisfacción del dueño (1-5).
- **Criterio para pasar a producción general:**
 - mediana de error por ambiente $\leq 5\%$, p90 $\leq 10\%$, total útil $\leq 3\%$;
 - $\geq 7/10$ completan sin ayuda;
 - tiempo mediano ≤ 15 min (3 ambientes).
- **Comparación de referencia (opcional, muy recomendada):** en 3 de las 10 propiedades, capturar también con **CubiCasa** (app gratuita, plano LITE) y comparar precisión. Da una vara externa y ayuda a decidir si conviene integrar en vez de construir alguna pieza.

15.4 Criterio de salida de la Fase 2

- Tour 360: en 10 propiedades, todos los ambientes navegables y los enlaces de puerta en la dirección correcta ($\pm 15^\circ$).
- Carga del primer panorama en menos de 3 s en 4G.
- Dollhouse sin ambientes superpuestos ni huecos mayores a 20 cm.
- Splat (si aplica): aprobación visual del asesor en $\geq 7/10$; si no, se queda en el Nivel 1+2.

16. Plan de implementación por fases y ciclos de corrección

Estimaciones para una persona full-time (Nico) con apoyo puntual. Ajustar después de la Fase 0.

Fase	Duración estimada	Entregables	Criterio de salida
F0 · Saneamiento y medición	1-2 semanas	Cron del pipeline legado; nginx /p/ y client_max_body_size; fix del contrato rooms front/back; excluir "Plano generado"; debounce de complete/; evento link_open server-side ; capture_stats básico	El link actual funciona de punta a punta en prod y se mide la apertura
F1a · Captura	3-4 semanas	App captura (modelos, API, prefirado S3, eventos); Capture App P0-P11 con disparo automático, QC, cola offline, modo simple; server re.bercovich.com; S3 CORS/IAM; stitching en vivo del 1er ambiente	E2E verde; matriz de dispositivos OK; 3 capturas internas completas sin pérdida de fotos
F1b · Plano	4-6 semanas	Worker GPU (layout, escala, ensamblado); renderer de polígonos + PDF; esquema plan/2; pantalla de revisión con editor de cotas; notificaciones; golden set en CI	Golden set $\leq 6\%$ mediana por ambiente con capturas internas
Piloto F1	2 semanas	10 propiedades reales, informe de precisión y UX	Criterios de la sección 15.3
Corrección F1 (ciclos)	1-2 semanas por ciclo	Ver 16.1	
Lanzamiento F1	1 semana	Flag para todos los asesores, runbook, tablero	
F2a · Tour 360	2-3 semanas	Nodos y enlaces, visor PSV bundleado, embed en ficha, /t/<slug>, reproceso de capturas existentes	Sección 15.4
F2b · Dollhouse	2-3 semanas	GLB texturizado, visor three.js, transición maqueta ↔ 360	

Fase	Duración estimada	Entregables	Criterio de salida
F2c · Splat (opcional)	3-4 semanas	Poses + gsplat + Spark, difuminado de rostros, gate de calidad	Decisión basada en calidad y costo

16.1 Cómo se trabajan las "fases de corrección"

1. **Recolección:** cada problema del piloto o de producción se carga como issue en `bercobackoffice` con la etiqueta `captura` y el `session_id`. Desde el backoffice se puede abrir la sesión y ver tomas, eventos y jobs.
2. **Clasificación:** - UX (el dueño no entendió o se trabó): se mira el embudo y los `shot_retry`; - captura (faltan o son malas las tomas): se ajustan umbrales de QC y protocolo, **sin redeploy** vía `CAPTURE_QC_CONFIG` y `CAPTURE_FLAGS`; - pipeline (error de medición): se agrega el caso al golden set; - infraestructura.
3. **Corrección y reproceso:** cada fix de pipeline se valida contra el golden set en CI y después se hace `reprocess_captures --stage <etapa>` sobre las sesiones afectadas. El asesor recibe el plano nuevo como versión n+1.
4. **Cadencia:** ciclos de 1 semana durante el piloto; cada ciclo termina con un reporte corto: métricas antes/después, cambios, sesiones reprocesadas.
5. **Versiónado:** `protocol_version` (cliente) y versiones de modelos (worker) quedan en cada sesión y en cada job. Así se sabe con qué se generó cada plano.

17. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Precisión insuficiente sin LiDAR (paredes blancas, baños chicos, contraluz)	Alto	Referencia de escala obligatoria sugerida; esquinas para multi-vista; confianza + revisión humana; golden set; plan B: "modo preciso" LiDAR o CubiCasa en casos rojos
El dueño abandona (captura larga)	Alto	Disparo automático; progreso visible; reanudable; recordatorios; medir el embudo y recortar tomas si el piloto lo indica
Safari limita la resolución de la cámara o los sensores	Medio	Resolución suficiente para el layout; modo simple; validar en la matriz antes del piloto
Links abiertos desde el navegador interno de WhatsApp	Medio	Detección + botón "Abrir en Safari/Chrome" + instrucción en el mensaje
Subidas pendientes al cerrar Safari	Medio	Subir durante la captura; P10 bloquea con aviso; recordatorio automático
Arranque en frío o costo de la GPU	Bajo	Apagado por inactividad; spot para F2; el volumen actual es bajo
Licencias de pesos	Medio	Tabla 14.3, <code>LICENSES.md</code> , revisión antes de cada cambio de modelo
Paralaje insuficiente para el splat	Medio (solo F2c)	Flag de barrido de video; F2 entrega igual el tour y el dollhouse
Privacidad (personas, objetos personales)	Medio	Textos de captura, difuminado de rostros, publicación solo con aprobación

18. Decisiones abiertas

1. **Ventanas en el plano v1:** ¿detectarlas (más trabajo) o dibujarlas solo si el revisor las marca? Propuesta: marcado manual en la v1.
2. **Segundo giro en ambientes grandes:** ¿regla automática (estimación en vivo de profundidad) o que lo decida el dueño con "es grande"? Propuesta: el dueño en la v1 y automático después del piloto.
3. **Bucket:** ¿prefijo en el bucket existente o bucket dedicado? Depende de si prod ya usa S3 (verificar `AWS_STORAGE_BUCKET_NAME` en el `.env` del EC2).
4. **Worker GPU:** EC2 `g6.xlarge` on-demand contra serverless. Propuesta: EC2 para el piloto.
5. **Modo preciso con LiDAR** (iPhone Pro, RoomPlan) como App Clip o app: ¿vale la pena después del piloto? Depende del error medido en los casos rojos.
6. **Coexistencia con el link actual:** ¿se reemplaza "Link de carga" por completo o conviven? Propuesta: reemplazar el botón en el backoffice y mantener `/p/` solo para links ya enviados, hasta que venzan.
7. **Publicación en portales:** formato de embed o URL del tour que aceptan Zonaprop, Argenprop y MercadoLibre (investigar en F2a).

19. Anexos

19.1 Textos de pantalla (primera versión, voseo)

- **P0:** "Hola {nombre} 🖐️ Vamos a armar el plano de tu propiedad en {dirección} con tu celular. Te guiamos paso a paso y lleva unos 15 minutos. Antes de empezar: prendé todas las luces, abrí cortinas y puertas interiores."
- **P2:** "Para guiarte necesitamos usar la cámara y el movimiento del celular. No grabamos audio ni tu ubicación."
- **P4:** "Parate adentro, de espaldas a la puerta de entrada, y apuntá hacia el interior. La foto se saca sola."
- **P6:** "Parate en el centro. Celular vertical a la altura del pecho. Girá despacio sobre tu lugar, moviendo los pies. Nosotros sacamos las fotos."
- **P7:** "Andá a un rincón y apuntá al rincón de enfrente. (1 de 4)"
- **P8:** "Poné una hoja A4 en el piso y sacale una foto desde arriba. Nos ayuda a que las medidas sean exactas."
- **P9:** "Parate en el marco de la puerta. Una foto hacia {ambiente anterior} y otra hacia {ambiente siguiente}."
- **P10:** "¡Muy bien! Revisá que estén todos los ambientes."
- **P11:** "¡Listo! Estamos armando tu plano. Te avisamos por WhatsApp."
- **Mensajes de QC:** "Más despacio" · "Enderezá el celular" · "Está oscuro, prendé la luz" · "Salió movida, quedate quieto" · "Sacá el dedo de la cámara" · "Mucha luz de la ventana, probá correr la cortina".

19.2 Referencias técnicas

- HorizonNet: <https://github.com/sunset1995/HorizonNet>
- LGT-Net: <https://github.com/zhigangjiang/LGT-Net>
- ZInD (Zillow Indoor Dataset): <https://github.com/zillow/zind>
- MapAnything: <https://github.com/facebookresearch/map-anything>
- Depth Anything 3: <https://github.com/ByteDance-Seed/Depth-Anything-3>
- MoGe: <https://github.com/microsoft/MoGe>
- VGGT: <https://github.com/facebookresearch/vggt>
- gsplat: <https://github.com/nerfstudio-project/gsplat>
- Spark (visor de splats): <https://github.com/sparkjsdev/spark>
- Photo Sphere Viewer (virtual tour): <https://photo-sphere-viewer.js.org/plugins/virtual-tour.html>
- COLMAP: <https://github.com/colmap/colmap>
- Permiso de sensores en iOS: https://developer.mozilla.org/en-US/docs/Web/API/DeviceOrientationEvent/requestPermission_static
- Layouts 360 (listado): <https://github.com/zhanght021/awesome-3D-Room-Layout-Estimation>
- CubiCasa (referencia comercial): <https://www.cubi.casa>
- Precisión declarada por CubiCasa: <https://armls.com/how-accurate-are-cubicasa-floorplan-scans>

19.3 Checklist de puesta en producción (F1)

- ☐ F0 desplegado; `link_open` visible en el tablero
- ☐ DNS y TLS de `re.bercovich.com` apuntando al server nuevo de nginx
- ☐ Permissions-Policy y CSP del host de captura verificadas (cámara y sensores OK; backoffice sigue en `camera=()`)
- ☐ Bucket, CORS, lifecycle e IAM creados; PUT desde el celular verificado
- ☐ Migraciones `captura` aplicadas; crons instalados (`crontab -l / etc/cron.d`)
- ☐ Worker GPU: imagen publicada, instancia creada, arranque y apagado automáticos probados, token rotado
- ☐ Golden set pasando en CI
- ☐ Matriz de dispositivos completada
- ☐ Runbook `deploy/RUNBOOK_CAPTURA.md` escrito (deploy, reproceso, worker, rollback)
- ☐ Feature flag solo para los asesores del piloto
- ☐ Tablero de embudo y alertas activos

19.4 Glosario

- **Giro 360 / panorama equirectangular:** imagen 2:1 que representa toda la esfera vista desde un punto.
- **Layout:** forma del ambiente (piso, paredes, techo) inferida desde un panorama.
- **Profundidad métrica:** distancia real en metros a cada píxel.
- **Paralaje:** diferencia entre vistas tomadas desde puntos distintos; necesaria para reconstruir 3D con volumen.
- **Dollhouse:** vista de maqueta 3D de la propiedad sin techo.
- **Gaussian splatting (splat):** técnica de reconstrucción 3D fotorrealista navegable en el navegador.
- **Superficie útil / cubierta / semicubierta:** interior entre muros / incluye muros / espacios techados abiertos (balcones).